

## # 05. GitHub 협업 규칙 (GitHub Collaboration Guidelines)

### 05. GitHub 협업 규칙 (GitHub Collaboration Guidelines)

#### 문서 정보

버전: v1.0 | 작성일: 2026-05-03 | 상태: 초안 | 작성자: 경북대학교 AI/빅데이터 전문가 양성 과정 — TBD팀

#### 한 줄 요약

학생 팀 + 발주처 협업 + 다양한 GitHub 숙련도를 모두 고려해, 충돌 없이 코딩하고 발표 시 “협업 체계까지 갖춘 팀”이라는 인상을 주는 표준 + CI + Branch Protection + 린터 통합 규칙.

#### 목차

- 1. 폴더 구조 표준
- 2. 브랜치 전략 (Git Flow 변형)
- 3. 커밋 메시지 컨벤션 (Conventional Commits)
- 4. Pull Request (PR) 규칙
- 5. Issue 관리
- 6. .gitignore 표준
- 7. 코드 린터·포매터 설정
- 8. GitHub Actions CI/CD
- 9. Branch Protection Rules
- 10. Secret Management
- 11. 팀원 온보딩 가이드 (Day 1)

## 1. 폴더 구조 표준

---

```

lemon-healthcare-project/
├── README.md
├── docs/                                # 기획·설계 문서 (10개)
│   ├── 01-project-overview.md
│   ├── 02-background-problem.md
│   ├── ...
│   └── 10-compliance-checklist.md
├── backend/                             # Python 백엔드 (FastAPI)
│   ├── src/
│   │   ├── algorithms/                 # v1~v4, 7-step 산출식
│   │   ├── ocr/                       # 영양제 OCR 파이프라인
│   │   ├── nutrition/                 # KDRIs 룩업 + 결핍 진단
│   │   ├── prediction/               # 체중 예측
│   │   ├── activity/                 # 활동점수
│   │   ├── api/                      # FastAPI 라우터
│   │   ├── models/                   # Pydantic 스키마, DB 모델
│   │   └── utils/
│   ├── tests/
│   │   ├── unit/
│   │   ├── integration/
│   │   └── fixtures/
│   ├── requirements.txt
│   ├── pyproject.toml                 # Black, Ruff, mypy 설정
│   └── README.md
├── mobile/                              # Flutter 모바일 앱
│   ├── lib/
│   │   ├── main.dart
│   │   ├── screens/                  # UI 화면
│   │   ├── widgets/                 # 재사용 위젯
│   │   ├── services/                # API 호출, HealthKit
│   │   ├── models/                  # 데이터 모델
│   │   └── utils/
│   ├── ios/
│   ├── android/
│   ├── test/
│   ├── pubspec.yaml
│   ├── analysis_options.yaml         # Dart 린터 설정
│   └── README.md
├── data/                                # 정적 데이터 (Git LFS 또는 별도 관리)
│   ├── kdris_2020.csv                # KDRIs 룩업 테이블
│   ├── food_db_sample.json
│   └── README.md                     # 출처·라이선스 명시
├── notebooks/                           # 실험·EDA 노트북
│   ├── 01_kdris_eda.ipynb
│   ├── 02_ocr_test.ipynb
│   └── README.md
├── scripts/                             # 보조 스크립트
│   ├── seed_database.py
│   └── README.md
├── .github/
└── PULL_REQUEST_TEMPLATE.md

```

```

|   |   |   | ISSUE_TEMPLATE/
|   |   |   | |   |   |   | bug_report.md
|   |   |   | |   |   |   | feature_request.md
|   |   |   | |   |   |   | question.md
|   |   |   | |   |   |   |
|   |   |   | |   |   |   | CODEOWNERS          # 코드 영역별 리뷰어 자동 지정
|   |   |   | |   |   |   | workflows/          # GitHub Actions
|   |   |   | |   |   |   | |   |   |   | ci-backend.yml
|   |   |   | |   |   |   | |   |   |   | ci-mobile.yml
|   |   |   | |   |   |   | |   |   |   | ci-docs.yml
|   |   |   | |   |   |   |
|   |   |   | |   |   |   |
|   |   |   | |   |   |   | .gitignore
|   |   |   | |   |   |   | .gitattributes      # Git LFS 설정 (필요 시)
|   |   |   | |   |   |   | .pre-commit-config.yaml # pre-commit 후 설정
|   |   |   | |   |   |   | .editorconfig      # 에디터 통일 설정
|   |   |   | |   |   |   | LICENSE           # 라이선스 (MIT 권장)

```

## 핵심 원칙

- **backend/** **mobile/** 분리: Python과 Dart는 의존성·언어가 완전히 다르므로 명확히 격리
- **docs/** 최상위: 기획 문서는 코드와 별개로 빠르게 접근 가능해야 함
- **data/** 는 Git LFS 권장: KDRIs CSV 외에 큰 파일(이미지 데이터셋)은 LFS

## 2. 브랜치 전략 (Git Flow 변형)

학생 팀에 적합한 단순화된 Git Flow 를 사용합니다.

### 2.1 브랜치 종류

```

main (production)
  ↑
  | (release PR — 발표/시연 시점에만 머지)
  |
develop (integration)
  ↑
  | (PR 머지)
  |
feature/<영역>-<기능>      hotfix/<이슈번호>-<설명>
bugfix/<이슈번호>-<설명>
docs/<문서번호>-<설명>

```

| 브랜치       | 역할           | 머지 대상          | 보호 수준                    |
|-----------|--------------|----------------|--------------------------|
| main      | 발표·시연용 안정 버전 | 직접 푸시 금지       | 🔴 최고 (Branch Protection) |
| develop   | 통합 개발 브랜치    | feature 머지 대상  | 🟡 중간 (CI 통과 + 1 리뷰)      |
| feature/* | 새 기능 개발      | develop으로 PR   | —                        |
| bugfix/*  | 버그 수정        | develop으로 PR   | —                        |
| hotfix/*  | main 긴급 수정   | main + develop | —                        |
| docs/*    | 문서 작업        | develop으로 PR   | —                        |

## 2.2 브랜치 네이밍 규칙

형식: <type>/<scope>-<short-description>

예시:

```
feature/algo-v1-step-score
feature/ocr-pipeline-mvp
feature/mobile-healthkit-connect
bugfix/123-bmr-calculation-error
hotfix/456-api-key-leak
docs/06-tech-stack-update
```

규칙: - 모두 **소문자** 사용 - 단어 구분은 **하이픈(-)** 사용 (언더스코어 X) - 짧고 명확하게 (40자 이하 권장) - 이슈 번호가 있으면 포함

## 2.3 브랜치 라이프사이클

1. develop 브랜치 최신화 → `git pull origin develop`
2. feature 브랜치 생성 → `git checkout -b feature/algo-v1-step-score`
3. 작업·커밋
4. push → `git push -u origin feature/algo-v1-step-score`
5. GitHub에서 PR 생성 (target: develop)
6. CI 통과 + 리뷰 통과
7. Squash & Merge (커밋 정리)
8. 브랜치 삭제 (자동 또는 수동)

### 3. 커밋 메시지 컨벤션 (Conventional Commits)

#### 3.1 형식

```
<type>(<scope>): <subject>

<body>

<footer>
```

#### 3.2 Type 종류

| Type     | 의미                | 예시                                                |
|----------|-------------------|---------------------------------------------------|
| feat     | 새 기능 추가           | feat(algo): add v1 step score calculation         |
| fix      | 버그 수정             | fix(ocr): handle empty supplement label edge case |
| docs     | 문서만 변경            | docs(readme): update setup instructions           |
| style    | 코드 스타일 변경 (포맷터 등) | style(backend): apply black formatter             |
| refactor | 리팩토링 (기능 변화 없음)   | refactor(api): split nutrition router             |
| perf     | 성능 개선             | perf(prediction): cache BMR calculation           |
| test     | 테스트 추가/수정         | test(algo): add edge cases for BMI categories     |
| chore    | 빌드, 설정 등          | chore(deps): upgrade fastapi to 0.110             |
| ci       | CI 설정 변경          | ci: add docs lint workflow                        |

#### 3.3 Scope (선택적)

영역을 명시할 수 있습니다.

| Scope     | 영역                   |
|-----------|----------------------|
| algo      | 알고리즘 (v1~v4, 7-step) |
| ocr       | 영양제 OCR              |
| nutrition | 영양 분석                |
| mobile    | Flutter 앱            |
| api       | FastAPI 라우터          |
| db        | 데이터베이스               |
| docs      | 문서                   |
| ci        | CI/CD                |

### 3.4 좋은 커밋 vs 나쁜 커밋

#### ✅ 좋은 예시

```
feat(algo): add v4 chronic disease weighting
```

Implement v4 score calculation that applies disease-specific multipliers (diabetes +0.10, hypertension +0.10, etc.) capped at 1.3.

Closes #42

#### ❌ 나쁜 예시

|                 |                     |
|-----------------|---------------------|
| update          | ← 무엇을 업데이트?         |
| fix bug         | ← 어떤 버그?            |
| WIP             | ← 작업 중이면 push 자제    |
| ㅁ ㄴ ㄹ ㅁ ㄴ ㄹ ㅁ ㄴ | ← 한글 키 실수 그대로       |
| asdf            | ← 의미 없음             |
| "수정함"           | ← 한국어도 OK이지만 구체적이어야 |

### 3.5 한국어 커밋 메시지 (선택)

영어가 부담스럽다면 한국어도 허용. 단 형식은 동일:

```
feat(algo): v1 활동점수 산출 함수 구현
fix(ocr): 빈 영양제 라벨 입력 시 에러 처리
docs(readme): 설치 가이드 업데이트
```

💡 **권장:** 영어를 기본으로 하되, 팀 회의에서 한국어 사용을 합의해도 무방. 단, **혼용은 피할 것** (한 PR 내에서 영어/한국어가 섞이면 가독성 ↓).

## 4. Pull Request (PR) 규칙

### 4.1 PR 생성 전 체크리스트

PR 작성자가 먼저 확인: - [ ] develop 브랜치를 머지/리베이스해 최신 상태 - [ ] 로컬에서 린터·테스트 모두 통과 - [ ] 커밋 메시지가 컨벤션에 맞음 - [ ] 관련 이슈 번호를 PR 설명에 명시 - [ ] 스크린샷/데모 (UI 변경 시) - [ ] 새 의존성 추가 시 이유 설명

### 4.2 PR 제목 규칙

커밋 메시지와 동일한 형식:

```
feat(algo): add v1 step score calculation
```

### 4.3 PR 템플릿

`.github/PULL_REQUEST_TEMPLATE.md` 파일을 두면 PR 생성 시 자동으로 채워집니다. (별도 파일로 제공)

### 4.4 코드 리뷰 규칙

| 항목             | 규칙                                    |
|----------------|---------------------------------------|
| 최소 리뷰어 수       | 1명 (develop 머지) / 2명 (main 머지)        |
| 리뷰 응답 시간       | 영업일 기준 24시간 이내                        |
| 자기 PR 머지 가능 시점 | 모든 리뷰어 Approve + CI 통과 후              |
| 리뷰 시 사용 언어     | “Approve / Request Changes / Comment” |

### 4.5 리뷰 코멘트 톤 가이드

| 상황    | ❌ 나쁜 표현         | ✅ 좋은 표현                                                   |
|-------|-----------------|-----------------------------------------------------------|
| 개선 제안 | “이거 왜 이렇게 했어요?” | “여기를 X 방식으로 바꾸면 Y 이점이 있을 것 같아요. 어떻게 생각하세요?”               |
| 버그 발견 | “여기 버그임”        | “여기서 N=0일 때 ZeroDivisionError 발생할 것 같습니다. 가드를 추가하면 어떨까요?” |
| 칭찬    | (생략하지 말 것)      | “👍 이 부분 깔끔하네요!”                                           |

💡 **원칙:** 사람을 비판하지 말고 코드를 비판한다. 항상 “왜 그렇게 했는지”가 아니라 “이렇게 하면 어떨지”의 톤으로.



## 4.6 머지 방식

- **Squash and Merge** (권장): feature 브랜치의 여러 커밋을 한 개로 압축
- **Rebase and Merge**: 커밋 히스토리를 살릴 가치가 있을 때만
- **Merge Commit**: 사용 X (히스토리 복잡)

## 4.7 PR 머지 후 작업

1. feature 브랜치 삭제 (GitHub UI에서 자동 옵션 사용)
2. 로컬에서도 정리:

```
git checkout develop
git pull origin develop
git branch -d feature/algo-v1-step-score
git remote prune origin
```

## 5. Issue 관리

### 5.1 Issue 생성 시점

- 새로운 기능을 시작하기 전 (Feature Request)
- 버그를 발견했을 때 (Bug Report)
- 질문이 있을 때 (Question)
- 리팩토링 또는 기술 부채 (Tech Debt)

### 5.2 Issue 라벨 시스템

#### 카테고리 라벨

- `type:bug` 🚫
- `type:feature` 🟢
- `type:docs` 🔵
- `type:refactor` 🟡
- `type:question` 🟣

#### 우선순위 라벨

- `priority:critical` 🚨 (즉시 처리)
- `priority:high` 🔥
- `priority:medium` 🌡️
- `priority:low` ❄️

## 영역 라벨

- `area:backend`
- `area:mobile`
- `area:algorithm`
- `area:ocr`
- `area:docs`
- `area:ci`

## 상태 라벨

- `status:in-progress`
- `status:blocked`
- `status:needs-review`
- `status:wontfix`

## 난이도 라벨 (학생 팀용)

- `good-first-issue` (신규 멤버용)
- `help-wanted`

## 5.3 Issue 템플릿

`.github/ISSUE_TEMPLATE/` 폴더에 다음 3개 템플릿 제공 (별도 파일): - `bug_report.md` - `feature_request.md` - `question.md`

## 5.4 Issue → PR 연결

PR 설명에 다음 키워드 사용 시 머지 시 자동으로 이슈 close:

```
Closes #42
Fixes #43
Resolves #44
```

## 6. .gitignore 표준

Python + Flutter + 일반 개발 환경 모두 커버하는 통합 .gitignore 사용 (별도 파일).

핵심 무시 항목: - Python: `__pycache__`, `*.pyc`, `.venv`, `*.egg-info` - Flutter/Dart:

`build/`, `.dart_tool/`, `*.g.dart`, `Pods/` - 환경 변수: `.env`, `.env.local` - IDE: `.vscode/`, `.idea/` (단 일부 공유 설정은 예외) - OS: `.DS_Store`, `Thumbs.db` - 데이터: `data/raw/`, `*.db`, `*.sqlite3` - 모델 가중치: `*.h5`, `*.pth`, `models/checkpoints/`

⚠️ **절대 금지:** API 키, 비밀번호, 사용자 데이터를 커밋하지 말 것. 한 번 커밋되면 git history에 영구 보존됨 (force-push 해도 fork에서 복원 가능).

## 7. 코드 린터·포매터 설정

### 7.1 Python (Backend)

#### 도구 스택

| 도구     | 역할             | 설정 파일          |
|--------|----------------|----------------|
| Black  | 코드 포매터         | pyproject.toml |
| Ruff   | 린터 + import 정렬 | pyproject.toml |
| mypy   | 타입 체크          | pyproject.toml |
| pytest | 테스트 러너         | pyproject.toml |

#### pyproject.toml 예시 (backend/pyproject.toml)

```
[tool.black]
line-length = 100
target-version = ["py311"]

[tool.ruff]
line-length = 100
target-version = "py311"
select = ["E", "F", "I", "N", "W", "UP"]
ignore = ["E501"] # line-length는 black이 처리

[tool.mypy]
python_version = "3.11"
strict = true
ignore_missing_imports = true

[tool.pytest.ini_options]
testpaths = ["tests"]
python_files = "test_*.py"
addopts = "-v --cov=src --cov-report=term-missing"
```

## 사용법

```
# 포매팅
black src tests

# 린트
ruff check src tests
ruff check src tests --fix # 자동 수정

# 타입 체크
mypy src

# 테스트
pytest
```

## 7.2 Dart/Flutter (Mobile)

### 도구 스택

| 도구            | 역할          |
|---------------|-------------|
| dart format   | 코드 포맷터 (내장) |
| dart analyze  | 정적 분석 (내장)  |
| flutter_lints | 추가 린트 규칙    |

### mobile/analysis\_options.yaml 예시

```
include: package:flutter_lints/flutter.yaml

analyzer:
  errors:
    invalid_annotation_target: ignore
  exclude:
    - "**/*.g.dart"
    - "**/*.freezed.dart"

linter:
  rules:
    - prefer_const_constructors
    - prefer_const_literals_to_create_immutables
    - avoid_print
    - sort_child_properties_last
    - use_key_in_widget_constructors
```

## 사용법

```
flutter format lib test
flutter analyze
flutter test
```

## 7.3 Pre-commit Hooks

커밋 전 자동으로 린터/포맷터 실행. 로컬에서 실수를 막아 CI 시간을 절약.

`.pre-commit-config.yaml` (별도 파일)

설치 방법 (팀원 온보딩 시):

```
pip install pre-commit
pre-commit install
```

이후 모든 `git commit` 이 자동으로 린트 검사를 통과해야 진행됨.

## 8. GitHub Actions CI/CD

### 8.1 CI 목표

PR이 develop/main에 머지되기 전에 다음을 자동 검증: - 코드 스타일 (black, dart format) - 린트 (ruff, dart analyze) - 타입 체크 (mypy) - 단위 테스트 (pytest, flutter test) - 빌드 가능 여부

### 8.2 워크플로 구성

세 개 워크플로 파일을 분리해 효율적으로 운영:

`ci-backend.yml` (백엔드 변경 시만 실행)

- 트리거: `backend/**` 변경 시
- Python 3.11 환경
- black -check / ruff / mypy / pytest

`ci-mobile.yml` (모바일 변경 시만 실행)

- 트리거: `mobile/**` 변경 시
- Flutter 안정 버전 환경
- dart format -output=none -set-exit-if-changed
- flutter analyze
- flutter test

`ci-docs.yml` (문서 변경 시만 실행)

- 트리거: `docs/**`, `*.md` 변경 시
- 마크다운 린트 (markdownlint)
- 깨진 링크 체크

별도 워크플로 파일들은 함께 제공됩니다.

## 8.3 CI 실패 시 대응

1. PR 페이지에서 빨간 X 클릭 → "Details" 클릭
2. 어떤 단계에서 실패했는지 확인 (formatting / lint / test)
3. 로컬에서 동일 명령 실행하여 재현
4. 수정 후 push → CI 자동 재실행

## 8.4 CI 캐싱

빌드 시간을 단축하기 위해: - pip 캐시 (Python) - pub 캐시 (Flutter) - gradle 캐시 (Android 빌드)

설정은 GitHub Actions의 `actions/cache@v4` 사용.

# 9. Branch Protection Rules

## 9.1 main 브랜치 (최고 보호)

GitHub Settings → Branches → Add rule:

| 규칙                                                               | 설정                               |
|------------------------------------------------------------------|----------------------------------|
| Require a pull request before merging                            | ✓                                |
| Require approvals                                                | ✓ 2명                             |
| Dismiss stale pull request approvals when new commits are pushed | ✓                                |
| Require review from Code Owners                                  | ✓                                |
| Require status checks to pass before merging                     | ✓                                |
| Require branches to be up to date before merging                 | ✓                                |
| Required status checks                                           | ci-backend , ci-mobile , ci-docs |
| Require conversation resolution before merging                   | ✓                                |
| Require signed commits                                           | (선택)                             |
| Require linear history                                           | ✓                                |
| Do not allow bypassing the above settings                        | ✓                                |

## 9.2 develop 브랜치 (중간 보호)

| 규칙                                           | 설정         |
|----------------------------------------------|------------|
| Require a pull request before merging        | ✓          |
| Require approvals                            | ✓ 1명       |
| Require status checks to pass before merging | ✓          |
| Required status checks                       | 변경된 영역의 CI |

## 9.3 CODEOWNERS 파일

`.github/CODEOWNERS` 로 영역별 자동 리뷰어 지정:

```
# 기본 리뷰어
*                @팀장

# 영역별 리뷰어
/backend/        @백엔드담당
/mobile/         @모바일담당
/docs/           @문서담당
/.github/        @팀장
```

# 10. Secret Management

## 10.1 절대 커밋하지 말아야 할 것

- ✗ API 키 (Google Cloud Vision, OpenAI, Anthropic, ...)
- ✗ 데이터베이스 비밀번호
- ✗ JWT secret key
- ✗ AWS/NCP 액세스 키
- ✗ 사용자 개인정보 샘플
- ✗ 의료기관 인증서
- ✗ Apple Developer 인증서·프로비저닝 프로파일
- ✗ Google Service Account JSON

## 10.2 환경변수 관리

### 로컬 개발

```
backend/.env (gitignore 됨)
GOOGLE_CLOUD_VISION_API_KEY=...
ANTHROPIC_API_KEY=...
DATABASE_URL=postgresql://...
```

`.env.example` 파일은 커밋 (값은 비워두고 키만):

```
backend/.env.example
GOOGLE_CLOUD_VISION_API_KEY=
ANTHROPIC_API_KEY=
DATABASE_URL=
```

## CI 환경

GitHub Repository → Settings → Secrets and variables → Actions → New repository secret

## 모바일 (API 키 보호)

- ❌ `apikey.dart` 에 하드코딩 X
- ✅ 백엔드 API를 거쳐서 호출하도록 설계 (앱에서 직접 외부 API 호출 X)

## 10.3 실수로 키 커밋했을 때

1. 즉시 해당 키 폐기·재발급 (Google Cloud Console 등에서)
2. 새 키로 교체
3. git history에서 제거: BFG Repo-Cleaner 또는 git filter-branch
4. force push (팀원에게 미리 공지)
5. 모든 팀원이 git pull --rebase

⚠️ 주의: 한 번 public repo에 커밋된 키는 즉시 폐기가 원칙. force-push 해도 GitHub fork·캐시·검색 봇이 이미 수집했을 가능성.

## 11. 팀원 온보딩 가이드 (Day 1)

새 팀원이 프로젝트에 합류할 때 따라야 할 1시간 체크리스트.

### Step 1. 저장소 클론

```
git clone https://github.com/<team>/lemon-healthcare-project.git
cd lemon-healthcare-project
```

### Step 2. 백엔드 환경 세팅

```
cd backend
python -m venv .venv
source .venv/bin/activate          # macOS/Linux
# .venv\Scripts\activate           # Windows
pip install -r requirements.txt
cp .env.example .env              # 그리고 값 채우기
pytest                            # 테스트 통과 확인
```



### Step 3. 모바일 환경 세팅

```
cd ../mobile
flutter pub get
flutter doctor          # iOS/Android 환경 확인
flutter test
```

### Step 4. Pre-commit 훅 설치

```
cd ..
pip install pre-commit
pre-commit install
```

### Step 5. 첫 커밋·PR 연습

**good-first-issue** 라벨이 붙은 이슈 중 하나를 골라: 1. feature 브랜치 생성 2. 작은 변경 (예: README 오타 수정)  
3. 커밋 → push → PR 4. CI 통과 확인 → 리뷰 요청

### Step 6. 도구 추천 (선택)

| 도구             | 역할                           |
|----------------|------------------------------|
| VS Code        | 통합 IDE (Python + Dart 모두 지원) |
| GitHub Desktop | Git 초보자용 GUI                 |
| Postman        | API 테스트                      |
| DBeaver        | DB 관리                        |

VS Code 추천 확장: - Python (Microsoft) - Pylance - Black Formatter - Ruff - Flutter (Dart Code) - GitLens -  
GitHub Pull Requests and Issues



### 변경 이력

| 버전   | 날짜         | 변경 사항                                             | 작성자 |
|------|------------|---------------------------------------------------|-----|
| v1.0 | 2026-05-03 | 초안 작성. 폴더구조·브랜치·커밋<br>·PR·CI·Branch Protection 통합 | TBD |



### 관련 문서

- [01. 프로젝트 개요](#)
- [06. 기술 스택](#)
- [08. 구현 계획](#)

## 함께 제공되는 설정 파일

---

- [.github/PULL\\_REQUEST\\_TEMPLATE.md](#)
- [.github/ISSUE\\_TEMPLATE/bug\\_report.md](#)
- [.github/ISSUE\\_TEMPLATE/feature\\_request.md](#)
- [.github/workflows/ci-backend.yml](#)
- [.github/workflows/ci-mobile.yml](#)
- [.gitignore](#)
- [.pre-commit-config.yaml](#)